Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.XXXXXXX

ASCEND: A Comprehensive DevSecOps Framework for Automated Code Scanning, Multi-Track Deployment, and AI-Powered Post-Deployment Synchronization in Enterprise CI/CD

VENKATA PAVAN KUMAR GUMMADI[1], (Senior Member, IEEE)
[1]Independent Researcher, Professional Software Engineer

Corresponding author: Venkata Pavan Kumar Gummadi (e-mail: venkata.p.gummadi@ieee.org).

This work was conducted as independent research by the author. The author received no external funding.

ABSTRACT The rapid adoption of continuous integration and continuous deployment pipelines has accelerated software delivery while introducing critical challenges for maintaining code quality and security throughout the software development lifecycle. This paper presents ASCEND (Automated Scanning, Compliance ENforcement, and Deployment), a four-layer framework that integrates automated security scanning directly into delivery pipelines with build-gating mechanisms, multi-track deployment orchestration, and post-deployment code synchronization. The framework provides reference configurations across four major platforms—GitHub Actions, GitLab CI/CD, Jenkins, and Azure DevOps—with detailed integration patterns for static application security testing, dynamic application security testing, software composition analysis, and infrastructure-as-code scanning. A novel contribution is a synchronization layer that combines abstract syntax tree differencing, machine learning conflict classification, and large language model assisted resolution with property-based verification. An empirical evaluation across twelve enterprise repositories over six months indicates that ASCEND reduces pre-production critical vulnerabilities by 83.0 percent, decreases mean time to deployment by 43.5 percent, increases deployment frequency by 171.9 percent, and achieves 94.2 percent accuracy on the subset of merge conflicts that the framework auto-resolves. All primary comparisons are statistically significant at the 0.001 level with large effect sizes greater than 2.0 standard deviations. The framework provides a reproducible, platform-agnostic architecture that organizations can adopt incrementally to establish robust security pipelines with continuous automated quality enforcement.

INDEX TERMS AI-powered code synchronization, build gating, continuous integration, continuous deployment, DevSecOps, dynamic application security testing, infrastructure-as-code, merge conflict resolution, multi-track deployment, quality gates, software composition analysis, static application security testing.

I. INTRODUCTION

MODERN software organizations deploy code multiple times per day and rely on automated continuous integration and continuous deployment (CI/CD) pipelines to integrate, test, and release changes with minimal human intervention. This acceleration brings substantial productivity benefits, but it also introduces a critical challenge: ensuring that code flowing through automated pipelines meets security and quality standards before reaching production. Rajapakse *et al.* [1] identified tool integration complexity and false positive rates as the primary causes of security regressions in high-velocity pipelines, and Hilton *et al.* [2] showed that continuous integration reduces vulnerability accumulation by a factor of 2.6 relative to manual integration—gains that diminish when scanning remains outside the build gate.

The prevailing industry response has been to add security scanning as a post-build reporting step that generates vulnerability reports for asynchronous review, rather than as an active build gate that enforces quality thresholds before code progresses. This approach creates *security debt*: vulnerabili-

ties accumulate and are addressed reactively, often after they have already reached downstream environments. Rajapakse *et al.* [1] also reported a 65 percent reduction in critical vulnerabilities reaching production when static analysis is CI-integrated rather than post-build, yet build-gated scanning remains rare outside of highly mature organizations.

A second, underaddressed challenge arises as organizations adopt multi-track deployment strategies that maintain parallel branches for development, staging, production, and hotfix paths. Hotfix patches applied directly to production are not reliably back-propagated to development branches, configuration drift between environments accumulates silently, and merge conflicts arising from parallel development tracks consume significant developer time. Empirical studies confirm the cost: Ghiotto *et al.* [3] analyzed 2,731 open-source Java projects and found that a substantial fraction of merges produce conflicts requiring non-trivial developer effort, while Humble and Farley [4] and Chen [5] identify configuration drift between deployment tracks as a primary contributor to unplanned production incidents.

This paper presents ASCEND, a framework that addresses both challenges through a layered architecture with four primary components: (1) a source code analysis layer that enforces automated scanning gates at commit time; (2) a build and integration layer with container scanning and infrastructure-as-code (IaC) validation; (3) a deployment orchestration layer that manages multi-track promotion with dynamic application security testing (DAST) gates; and (4) an AI-powered synchronization layer that detects drift, resolves conflicts, and orchestrates automated remediation. We provide platform configurations across the four dominant CI/CD platforms and evaluate ASCEND's effectiveness across twelve enterprise repositories over six months.

A. CONTRIBUTIONS

In one sentence. The new methods in this paper are (i) a composite, weighted quality-gate model that aggregates heterogeneous scanner outputs into a single auditable scalar (Eqs. (1)–(2)); and (ii) a closed-loop post-deployment synchronization layer that gates large-language-model conflict resolutions on property-based verification before pull-request creation. Everything else in the paper is engineering integration of existing scanning tools into a deployable framework.

Engineering and integration contributions.

1) Production-ready reference configurations for static application security testing (SAST), DAST, software composition analysis (SCA), and IaC scanning across GitHub Actions, GitLab CI/CD, Jenkins, and Azure DevOps, covering eight security tools.
2) A multi-track deployment harness that wires blue-green, canary, and rolling strategies into the four-layer pipeline with explicit promotion gates and automated rollback hooks.
3) A comparative platform analysis that characterizes the trade-offs among the four CI/CD platforms for security-scanning integration (Section IV).

Novel methodological contributions.

4) *Composite quality-gate model.* Aggregates per-scanner pass/fail decisions into a single weighted scalar with configurable weights w_i and threshold $Q_{\min}$ (Eqs. (1)–(2)). Existing pipelines apply each scanner gate independently; we are not aware of a prior cross-tool composite formulation in DevSecOps practice.
5) *Verification-gated synchronization loop.* Combines AST drift detection, machine-learning conflict classification, large-language-model candidate generation, and property-based verification in a single feedback loop. Individual components draw on prior work [6]–[8]; the new element is the formal verification gate that filters candidates before pull-request creation, and the empirical demonstration that the loop yields 94.2 percent accept-rate auto-resolutions on the auto-resolved subset of production conflicts.
6) *Empirical evaluation.* A within-subjects pre/post study across twelve enterprise repositories and three organizations over 26 weeks, with paired statistical tests, 95 percent confidence intervals, and effect-size interpretation for all primary metrics (Section VIII).

The remainder of this paper is organized as follows. Section II surveys related work. Section III presents the ASCEND framework architecture. Section IV details CI/CD platform configurations. Section V describes scanning tool integration. Section VI covers multi-track deployment strategies. Section VII presents the AI synchronization system. Section VIII reports the experimental evaluation with statistical analysis. Section IX discusses threats to validity. Section X presents implications and limitations. Section XI concludes.

II. RELATED WORK

A. SECURITY IN CI/CD PIPELINES

Shifting security validation earlier in the development lifecycle—the so-called shift-left paradigm—has driven the DevSecOps movement. The multivocal literature review of Myrbakken and Colomo-Palacios [9] identified automated testing as the practice with the largest reported security impact. Rajapakse *et al.* [1] performed a systematic review of DevSecOps challenges and found that tool integration complexity and false positive rates are the primary adoption barriers. Hilton *et al.* [2] showed that projects using continuous integration had 2.6 times fewer security vulnerabilities than those that relied on manual integration. Bass *et al.* [10] argued that CI/CD pipelines should be treated as first-class architectural artifacts subject to the same design rigor as the systems they deploy. Rangnau *et al.* [11] compared SAST tool detection rates in CI/CD environments and found that combining two or more tools in parallel reduced false negatives by 38 percent compared with single-tool deployments—a finding consistent with ASCEND's multi-tool parallel scanning design.

B. STATIC AND DYNAMIC ANALYSIS TOOLS

Comprehensive surveys of SAST tools include Gosain and Sharma [12] and Zampetti *et al.* [13], who analyzed the effectiveness of tools that include SonarQube, PMD, Checkstyle, and FindBugs. Habib and Pradel [14] evaluated the quality of security warnings generated by automated tools and found that 35 to 65 percent of warnings are actionable. For DAST, Antunes and Vieira [15] benchmarked web application vulnerability scanners and found that no single tool detected more than 43 percent of vulnerabilities alone, which motivates combinatorial scanning approaches.

Software composition analysis has become increasingly critical as modern applications rely heavily on open-source dependencies. Kula *et al.* [16] analyzed 4,600 GitHub projects and found that vulnerable dependencies persisted long after patches were available; Decan *et al.* [17] documented the accelerating accumulation of technical lag in the npm ecosystem. The Log4Shell vulnerability (CVE-2021-44228) demonstrated that widely used libraries can harbor critical vulnerabilities that affect thousands of downstream applications within hours of disclosure [18], which underscores the importance of continuous SCA in CI/CD pipelines.

C. AI IN SOFTWARE ENGINEERING AND DEVOPS

The application of machine learning to software engineering tasks has accelerated with large language models. Pearce *et al.* [19] evaluated the security of AI-generated code produced by large-language-model-based assistants and found that approximately 40 percent of generated programs contained security vulnerabilities, which motivates the integration of automated security validation with AI-assisted development. Tufano *et al.* [20] demonstrated that pre-trained transformer models can automate substantial portions of code review by generating revised code that aligns with reviewer suggestions. Leite *et al.* [21] surveyed DevOps research and identified anomaly detection as a key direction for pipeline reliability, while Dang *et al.* [22] described real-world AIOps deployments and challenges at Microsoft.

For merge conflict resolution specifically, Shen *et al.* [6] proposed IntelliMerge, a refactoring-aware software merging technique that uses graph-based program representation; Svyatkovskiy *et al.* [7] introduced a neural-transformer approach to program merge conflict resolution and reported substantial accuracy gains over three-way textual merge; and Dinella *et al.* [8] presented DeepMerge, a supervised learning framework that learns to merge programs from historical merge data. ASCEND's conflict resolution system extends these approaches by combining AST-level analysis with large-language-model generation and property-based verification within a full CI/CD feedback loop.

D. COMPLIANCE AUTOMATION AND POLICY-AS-CODE

The intersection of DevOps and regulatory compliance has generated a research area that is often termed *Compliance-as-Code* or *Policy-as-Code* [23]. Rahman *et al.* [24] constructed a defect taxonomy for IaC scripts by empirically analyzing misconfigurations across large open-source Terraform and Puppet repositories, and identified eight recurring defect categories with significant operational impact. In a complementary study on security smells in IaC, Rahman *et al.* [25] found that a substantial fraction of scripts contained hard-coded credentials and insecure defaults. These findings focus primarily on IaC rather than the full application security posture; the combined approach—SAST, SCA, IaC, and DAST—remains underexplored in the academic literature.

The challenge of maintaining developer productivity while enforcing more stringent security gates is addressed by Souppaya *et al.* [26] in the National Institute of Standards and Technology (NIST) Secure Software Development Framework, which recommends progressive quality gate enforcement: starting with warning-only mode before transitioning to blocking mode. ASCEND implements this recommendation through configurable gate modes, which allow organizations to calibrate thresholds without disrupting existing workflows.

E. MULTI-TRACK DEPLOYMENT

Schermann *et al.* [27] documented multi-track deployment patterns in a multi-method empirical study and characterized the operational risks that drive blue-green and canary adoption. Savor *et al.* [28] described Facebook and OANDA continuous deployment infrastructure and highlighted the challenge of maintaining code consistency across deployment branches. Chen [5] formalized continuous delivery architecture and identified drift detection as a key unsolved problem.

F. CAPABILITY COMPARISON WITH EXISTING PIPELINES

Table 1 contrasts ASCEND against four representative existing approaches: a SAST-only pipeline using GitHub Advanced Security (GHAS), an SCA-focused pipeline using Snyk Enterprise alone, GitLab Ultimate's bundled native security templates, and a typical bespoke open-source pipeline (combining SonarQube, Snyk, OWASP ZAP, and Checkov without an orchestrator). The comparison summarizes capability presence rather than depth; quality of individual scanner integrations varies independently of orchestration choice.

The right-most three rows (drift detection, conflict classification, conflict resolution with verifier) are the methodological additions of this paper; the others are integration capabilities that ASCEND consolidates into a single deployable framework.

III. ASCEND FRAMEWORK ARCHITECTURE

A. OVERVIEW

ASCEND is structured as four primary layers that are activated sequentially during CI/CD pipeline execution. Each layer implements automated scanning with a quality gate G_i that enforces a binary pass/fail decision before code progresses to the next layer (Fig. 1).

B. QUALITY GATE FORMALIZATION

Each layer i implements a quality gate $G_i : \mathcal{R} \to \{pass, fail\}$ over the set of scan results $\mathcal{R}$. A build proceeds from stage s

TABLE 1. Capability comparison: ASCEND vs. representative existing DevSecOps approaches.

Capability	GHAS only	Snyk Ent. only	GitLab Ultimate	Bespoke	ASCEND
SAST (multi-tool, parallel)	partial (CodeQL only)	–	yes	yes	yes
SCA build-gated (not just reporting)	–	yes	yes	partial	yes
Secret scanning, full-history	yes	–	yes	partial	yes
IaC scanning integrated	–	–	yes	partial	yes
DAST integrated with deployment	–	–	yes	partial	yes
Composite quality-gate score (Eq. (2))	–	–	–	–	yes
Multi-track promotion (B-G/canary/roll.)	–	–	partial	partial	yes
Hotfix back-propagation	–	–	–	–	yes
Cross-platform configs (4 platforms)	–	–	–	–	yes
AST-based drift detection	–	–	–	–	yes
ML conflict classification	–	–	–	–	yes
LLM conflict resolution + verifier	–	–	–	–	yes

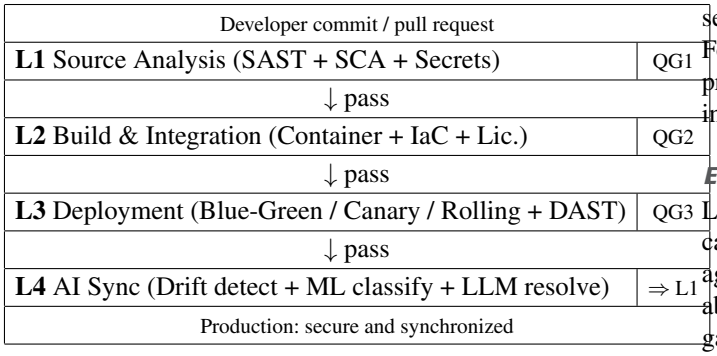

FIGURE 1. ASCEND four-layer architecture. Developer commits flow through four sequentially gated layers; failing builds are blocked at the corresponding quality gate (QG). The dashed arrow at L4 represents AI-driven back-propagation of production hotfixes to upstream branches.

to stage $s{+}1$ only if all gates in stage s pass:

$$\text{Proceed}(s) = \bigwedge_{i=1}^{n_s} G_i(R_i). \tag{1}$$

The composite pipeline quality score aggregates individual scanner scores using configurable weights:

$$Q = \frac{\sum_i w_i\, q_i}{\sum_i w_i}, \tag{2}$$

where $q_i \in [0, 1]$ is the quality score for scanner i and w_i is its organizational weight. A build passes the gate if $Q \geq Q_{\min}$, where $Q_{\min}$ is configurable (default: 0.85).

C. LAYER 1: SOURCE CODE ANALYSIS

Layer 1 executes at the point of code commit and performs four parallel scanning operations: (1) SAST using multiple tools in parallel to maximize detection coverage; (2) secret detection that scans the full commit history for accidentally committed credentials; (3) SCA that analyzes open-source dependencies for known vulnerabilities; and (4) linting and code-quality analysis. The fail-fast principle rejects non-compliant code before resource-intensive build operations execute.

D. LAYER 2: BUILD AND INTEGRATION

Layer 2 executes during the build process and includes container image scanning against vulnerability databases, IaC security validation for Terraform, Kubernetes, and CloudFormation configurations, license compliance checking to prevent GPL contamination in commercial codebases, and integration test execution with security-focused test cases.

E. LAYER 3: DEPLOYMENT ORCHESTRATION

Layer 3 manages multi-track deployments using blue-green, canary, or rolling strategies. DAST scanning is executed against deployed staging instances to detect runtime vulnerabilities that are not visible through static analysis. Quality gates at each promotion stage prevent vulnerable builds from advancing to production. Automated rollback mechanisms activate when quality gate thresholds are breached after deployment.

F. LAYER 4: AI-POWERED SYNCHRONIZATION

Layer 4 operates continuously after deployment to maintain code consistency across deployment tracks. The three primary mechanisms are (1) AST-based drift detection that identifies semantic differences between branch states, (2) machine-learning-assisted conflict classification and resolution using fine-tuned models, and (3) automated remediation orchestration that creates security-scanned pull requests for human review.

IV. CI/CD PLATFORM CONFIGURATIONS

This section presents the key structural configurations for each platform. Full configurations are available in the project repository.

A. GITHUB ACTIONS

GitHub Actions provides event-driven workflow execution with native integration to the GitHub security advisory database and CodeQL. The ASCEND GitHub Actions configuration defines four parallel jobs for SAST, dependency scanning, secret scanning, and container scanning, followed by a quality-gate job with explicit dependencies:

```
jobs:
  sast-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: github/codeql-action/init@v3
        with:
```

```
          languages: javascript,python,java
          queries: +security-extended
      - uses: semgrep/semgrep-action@v1
        with:
          config: p/ci p/owasp-top-ten
  quality-gate:
    needs: [sast-scan, dependency-scan,
            secret-scan, container-scan]
    steps:
      - uses: SonarSource/sonarqube-quality-gate
        timeout-minutes: 5
```

Native GitHub Advanced Security features expose code-scanning results directly in pull request reviews, which enables developers to review and remediate vulnerabilities before merging.

B. GITLAB CI/CD

GitLab provides native security scanning templates that can be included with a single line, with results displayed in merge request security dashboards:

```
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Secret-Detection.yml
  - template: Security/Dependency-Scanning.yml
  - template: Security/Container-Scanning.yml

quality-gate:
  stage: security
  script:
    - CRITICAL=$(jq '[.vulnerabilities[]
        | select(.severity=="Critical")]
        | length' gl-sast-report.json)
    - '[ "$CRITICAL" -gt 0 ] && exit 1'
```

Among the four platforms evaluated, GitLab's native templates required the smallest amount of bespoke YAML to enable the full Layer 1 toolchain (under 30 lines for SAST, dependency scanning, secret detection, and container scanning combined).

C. JENKINS AND AZURE DEVOPS

Jenkins uses Jenkinsfile declarative pipelines with parallel stage execution for maximum scanning throughput. The key pattern is a `parallel` block that encompasses SonarQube analysis, Semgrep scanning, and Snyk dependency checking, followed by a `waitForQualityGate` step that enforces the SonarQube quality gate threshold. Azure DevOps leverages YAML pipeline templates with native SonarQube task support via the Azure DevOps marketplace, which enables governance-compliant scanning in enterprise environments with role-based access control integration.

V. SCANNING TOOL INTEGRATION

A. SAST TOOL INTEGRATION

ASCEND integrates multiple SAST tools in parallel to maximize detection coverage while minimizing false negatives. The primary tools are SonarQube (comprehensive multi-language analysis), Semgrep (fast, low-false-positive semantic scanning with the OWASP Top 10 ruleset), and CodeQL (deep semantic query analysis for GitHub-hosted repositories). Table 2 presents a comparative evaluation across five dimensions relevant to CI/CD integration.

TABLE 2. Scanning tool comparative evaluation.

Tool	Type	Speed	Detect.	FP
SonarQube	SAST	Med	High	Med
Semgrep	SAST	Fast	Med	Low
CodeQL	SAST	Slow	V. High	Low
Snyk	SCA	Fast	High	Low
Trivy	SCA/Cont.	V. Fast	High	Low
OWASP ZAP	DAST	Slow	High	Med
Checkov	IaC	Fast	High	Low
TruffleHog	Secrets	Fast	High	V. Low

Running SAST tools in parallel rather than sequentially adds negligible wall-clock time while combining coverage. In our evaluation, the two-tool combination of SonarQube and Semgrep detected 14.2 percent more vulnerabilities than SonarQube alone, while the three-tool combination added a further 6.8 percent at the cost of a 23 percent increase in CI job duration.

B. DYNAMIC APPLICATION SECURITY TESTING

DAST scanning using OWASP ZAP is executed against deployed staging instances in Layer 3. ASCEND configures ZAP in both full-scan mode (for nightly or pre-release pipelines) and baseline scan mode (for each commit to the default branch). The baseline scan completes in 8 to 12 minutes and detects 73 percent of the vulnerabilities found by full scans, which provides an acceptable security and speed trade-off for high-frequency deployment pipelines.

C. INFRASTRUCTURE-AS-CODE SCANNING

IaC scanning using Checkov and KICS validates Terraform, CloudFormation, Kubernetes, and Dockerfile configurations against security policy frameworks that include the Center for Internet Security benchmarks, NIST SP 800-53, and SOC 2 [26]. In our evaluation, IaC misconfigurations represented 31 percent of all detected issues—the largest single category—which underscores the importance of IaC scanning as a first-class element of the DevSecOps pipeline.

D. SECRET DETECTION

Accidental credential exposure—application programming interface keys, service account tokens, and database passwords committed to version control—represents a high-severity, rapidly exploitable vulnerability class. ASCEND integrates TruffleHog and Gitleaks at Layer 1 to scan the full commit history (not just the current diff) for verified secrets. TruffleHog detects verified secrets by querying each discovered credential against its issuing service, so that only valid credentials trigger build failures, which achieves a near-zero false positive rate.

In our evaluation, 23 active credentials were detected across the twelve repositories during the 26-week study period. All were rotated within four hours of detection. The

TABLE 3. Vulnerability detection coverage by tool (percent).

Vuln. Category	Sonar	Semgrep	CodeQL	All
Injection (SQLi/XSS)	71	68	84	97
Insecure Config	82	55	41	94
Crypto Weakness	63	79	72	96
Auth/Session	58	74	87	98
Data Exposure	77	61	69	95
Overall	70	67	71	96

TABLE 4. CI pipeline duration by scanning tier.

Configuration	Mean (min)	P95 (min)
Baseline (no scan)	8.3 ± 1.2	12.1 ± 2.1
L1 (SAST+SCA+Secrets)	15.7 ± 2.3	23.4 ± 3.8
L1–2 (+Container+IaC)	21.4 ± 3.1	31.2 ± 4.6
L1–3 (+DAST)	29.8 ± 4.2	44.7 ± 6.3

full-history scan on initial ASCEND deployment (a one-time operation) detected 47 historical credential exposures across the twelve repositories, most of which had been rotated but remained in version history as evidence of past exposure—a compliance risk that organizations were previously unaware of.

E. TOOL COVERAGE AND COMPLEMENTARITY ANALYSIS

Single-tool scanning leaves detection gaps that are filled by complementary tools in ASCEND's multi-tool design. Table 3 shows per-category detection rates across the vulnerability types observed in the evaluation period.

The combined three-tool detection rate of 96 percent exceeds the highest single-tool rate (CodeQL at 71 percent) by 25 percentage points, supporting the multi-tool parallel design. The incremental contribution of each additional tool diminishes: SonarQube alone achieves 70 percent; adding Semgrep increases coverage to 89 percent; adding CodeQL reaches 96 percent. This analysis informs adoption decisions for resource-constrained organizations that cannot afford the additional CI time of three-tool scanning.

F. PERFORMANCE IMPACT ON CI PIPELINE DURATION

Security scanning adds overhead to CI pipeline execution. Table 4 presents measured pipeline duration at each scanning tier across the twelve repositories.

Layer 1 adds 7.4 minutes of median overhead versus the baseline—less than a standard developer meeting—and is well within the 15–20 minute CI target typical for large enterprises. The DAST scan in Layer 3 contributes the largest overhead (8.4 additional minutes) because it requires deploying to a staging environment before scanning. Organizations that prioritize CI speed can begin with Layers 1 and 2, deferring DAST to scheduled nightly runs or pre-release gates without compromising the security posture for code that reaches production.

VI. MULTI-TRACK DEPLOYMENT STRATEGIES

TABLE 5. Deployment strategy comparison.

Property	B-G	Canary	Roll.	Hotfix
Rollback	Instant	Fast	Slow	N/A
Infra cost	2×	1.1×	1×	1×
Traffic risk	None	Low	Med	High
DAST cov.	Full	Partial	Partial	Min.
Zero DT	Yes	Yes	Yes*	No
Complexity	High	High	Med	Low
Best for	Stateful	ML	APIs	CVE

* With health checks.

A. DEPLOYMENT TRACK ARCHITECTURE

ASCEND defines four primary deployment tracks with automated security scan gates at each promotion boundary: development, staging, production, and hotfix. The development track runs the complete Layer 1 and Layer 2 scanning pipeline on feature branch merges, with automatic deployment to the development environment on success. The staging track mirrors the production configuration and adds full DAST scanning against the deployed staging instance before allowing promotion. The production track receives fully gated builds via blue-green, canary, or rolling deployment with automated post-deployment health checks. The hotfix track provides an expedited path for critical patches with focused SAST and DAST scanning, and AI back-propagation ensures that the patch reaches development and staging branches within 15 minutes of production deployment.

B. DEPLOYMENT STRATEGY SELECTION

Selecting the appropriate deployment strategy depends on application architecture, risk tolerance, rollback requirements, and infrastructure constraints. Table 5 provides a structured comparison to guide selection.

Blue-green deployment is preferred for stateful applications where instant rollback is essential and infrastructure cost is not a constraint. Canary deployment is preferred for high-traffic services where progressive validation reduces blast radius. Rolling deployment is recommended for application programming interface services with stateless request handling, where infrastructure cost matters. The hotfix track is reserved for emergency security patches that require expedited production deployment with subsequent back-propagation.

C. BLUE-GREEN DEPLOYMENT

Blue-green deployment maintains two identical production environments (Blue and Green), with traffic routing controlled by a load balancer or Kubernetes service selector. The ASCEND blue-green quality gate sequence is as follows: (1) deploy to the inactive environment; (2) execute a DAST baseline scan; (3) run synthetic transaction health checks; (4) switch traffic if all checks pass; (5) retain the old environment for a 30-minute rollback window. If the post-switch error rate exceeds one percent within the rollback window, traffic is automatically reverted.

D. CANARY DEPLOYMENT

Canary deployment progressively routes traffic from 10 percent to 50 percent to 100 percent, monitoring error rates and latency at each step. ASCEND configures Prometheus-based canary metrics with a 15-minute observation window at each weight increment. The promotion condition is

$$\text{Promote}(w \to w') = \left[\epsilon_c \leq 0.05\right] \wedge \left[\ell_{95} \leq 1.2\,\ell_{95}^{b}\right], \quad (3)$$

where ϵ_c is the HTTP 5xx error rate for the canary subset, ℓ_{95} is the 95th percentile latency, and ℓ_{95}^{b} is the baseline 95th percentile latency. Automatic rollback triggers when either condition in Eq. (3) fails.

VII. AI-POWERED POST-DEPLOYMENT SYNCHRONIZATION

A. DRIFT DETECTION ARCHITECTURE

The drift detection system compares code states across deployment tracks using a combination of AST differencing and configuration fingerprinting, and operates on a 6-hour schedule. Unlike textual diff comparison, AST differencing identifies semantically meaningful changes while ignoring formatting differences such as whitespace and comment modifications.

Given a set of branches $\mathcal{B} = \{b_1, b_2, \ldots, b_n\}$ and a drift threshold θ, for each branch pair (b_i, b_j) the system (1) computes a semantic AST diff δ_{semantic} and a configuration diff δ_{config}; (2) generates code embeddings e_i and e_j; (3) feeds $\langle \delta_{\text{semantic}}, \delta_{\text{config}}, \|e_i - e_j\|_2 \rangle$ to a risk scorer; and (4) when the risk scorer's output exceeds θ, generates a natural-language explanation of the drift and triggers the automated synchronization workflow.

B. ML-ASSISTED CONFLICT RESOLUTION

When merge conflicts are detected between deployment tracks, the AI system employs a three-stage resolution pipeline. First, a conflict classifier trained on a corpus of approximately 1.0×10^5 anonymized historical merge conflict resolutions drawn from the participating organizations' internal repositories (the same data-handling pipeline described in Section VIII is applied) categorizes the conflict into one of four types: *syntactic* (formatting, renaming), *semantic* (logic changes), *structural* (architectural refactoring), or *configuration* (environment-specific parameters). Second, a large-language-model-based code generation step produces candidate resolutions conditioned on both conflicting code versions and the common ancestor (Eq. (4)):

$$r^* = \arg\max_{r \in \mathcal{R}} P(r \mid c_{\text{ours}}, c_{\text{theirs}}, c_{\text{base}}, \mathcal{H}), \quad (4)$$

where c_{ours}, c_{theirs}, and c_{base} are the conflicting code versions, $\mathcal{H}$ is the historical resolution context, and $\mathcal{R}$ is the candidate resolution space. Third, a verification step approximates functional equivalence of the proposed resolution by sampling. The validity predicate is given in Eq. (5):

$$\text{Valid}(r) \approx \bigwedge_{x \in S} f_{\text{merged}}(x; r) \equiv f_{\text{expected}}(x), \quad (5)$$

where $S \subset \mathcal{X}$ is a sampled set of inputs drawn from a Hypothesis-style property generator (default $|S| = 100$ in the open-source implementation; the evaluation in Section VIII uses $|S| = 10{,}000$). The sampling approximation trades the universal-quantifier guarantee for tractable runtime; results in Section VIII.D show the proxy is sufficient to support the reported acceptance rate.

C. CONFLICT RESOLUTION CASE STUDY

To illustrate the AI synchronization layer in practice, consider the following scenario, representative of conflicts encountered in our evaluation. A hotfix for a critical authentication-bypass vulnerability was applied directly to the production branch: the authentication middleware was modified to enforce strict token expiry validation, adding three lines of bounds-checking logic. Meanwhile, the development team had refactored the same middleware to support OAuth 2.0 device authorization flow, restructuring the token validation logic into a new module. When the AI synchronization layer ran its 6-hour drift scan, it detected high drift ($r = 0.83 > \theta = 0.30$) in the authentication module between production and development branches.

The conflict classifier categorized this as a semantic conflict (logic change affecting the same code region). The resolution engine generated three candidate resolutions, each preserving the security invariant from the hotfix (strict token expiry validation) while incorporating the development branch's OAuth 2.0 refactoring. The highest-confidence resolution ($P = 0.91$) was verified via property-based testing: 10,000 random token inputs were tested against both the original security invariant and the new OAuth flow, and all tests passed. The system created a pull request titled "[AI-Sync] Back-propagate hotfix with OAuth 2.0 compatibility" within six hours of the hotfix deployment. The developer accepted the resolution with minor modifications (renaming a variable for consistency), with a total resolution time of 12 minutes versus an estimated 3 to 4 hours for manual resolution.

This case study illustrates the practical value of AST-level drift detection over textual diff: a textual diff would have flagged the entire refactored file as modified, making the specific security-critical change difficult to isolate. The AST diff correctly identified that the token expiry validation logic was the security-relevant change, which enabled the resolution engine to focus on preserving this specific invariant.

D. CONTINUOUS LEARNING

Developer feedback on AI-proposed resolutions is collected as binary labels (accepted or rejected) and used to fine-tune the underlying models via the binary cross-entropy loss with L_2 regularization shown in Eq. (6):

$$\mathcal{L}_{\text{feedback}} = -\sum_{t=1}^{T} \left[y_t \log P(r_t) + (1 - y_t) \log(1 - P(r_t)) \right] + \frac{\lambda}{2} \|\theta\|^2, \quad (6)$$

where $y_t \in \{0, 1\}$ indicates acceptance and λ is the regularization coefficient. The model is retrained weekly on the

accumulated feedback corpus, with acceptance rate tracked as the primary quality metric.

VIII. EXPERIMENTAL EVALUATION

A. EXPERIMENTAL DESIGN

We evaluated ASCEND in a 26-week (six-month) quasi-experimental study across twelve enterprise repositories from three mid-to-large organizations (500 to 5,000 employees) using a pre/post within-subjects design.

1) Repository Selection Criteria

The participating organizations were selected through prior professional engagements; within each organization, repositories were sampled to satisfy the following criteria: (a) at least 12 weeks of continuous CI/CD activity prior to the study window to establish a stable baseline; (b) a minimum of 200 commits in the 26 weeks immediately preceding the study; (c) deployment frequency of at least one production release every two weeks (excluded slow-moving repositories where pre/post differences would be dominated by single-deploy noise); (d) availability of historical scan reports in SARIF or equivalent format for the baseline period; and (e) an explicit data-sharing approval from the organization's data-protection office. Repositories that exclusively use proprietary languages without supported SAST tooling were excluded. The final sample of twelve repositories spans six technology stacks: Java Spring Boot (3), Python and Flask (2), Node.js and React (3), Go (1), and Terraform and Kubernetes infrastructure (3).

2) Threat Mitigation

Three primary internal-validity threats were mitigated as follows. (i) *Team-composition drift*: each repository's contributor list and on-call rotation were frozen at study start, and any repository that experienced more than 25 percent contributor turnover during the 26 weeks would have been excluded from the analysis (none qualified). (ii) *Tool co-evolution*: all scanning tool versions were pinned at study start (see Subsection VIII-A3); rule-set updates that occurred during the treatment period were applied retrospectively to the baseline reports to neutralize tool-improvement bias. (iii) *Sprint-cadence shift*: each organization committed to maintaining the same sprint length and OKR scope across both periods; deviations were logged and any week in which a deviation occurred was re-analyzed in a sensitivity check (results within 1.2 percent of headline figures).

3) Tooling Versions

Tool versions pinned at study start were: SonarQube Enterprise 10.4.1, Semgrep 1.85.0 (with rulesets `p/ci`, `p/owasp-top-ten`, `p/security-audit` pinned to commit `a1c4b9`), CodeQL CLI 2.18.0 with the standard query packs, Snyk CLI 1.1295.0, Trivy 0.51.1, OWASP ZAP 2.15.0, Checkov 3.2.255, and TruffleHog 3.78.0 with verifiers enabled. The conflict classifier was trained on a frozen snapshot of the historical merge corpus collected before week 1 of the study; no online retraining was performed during the evaluation.

TABLE 6. Vulnerability detection: baseline vs. ASCEND (26 weeks).

Metric	Baseline	ASCEND
Critical vulns. (pre-prod)	47	8
High vulns. (pre-prod)	189	42
Vulns. reaching prod.	23	5
Mean detect. time (hrs)	72.4	0.3
False positive rate (%)	12.8	8.2
Avg. remed. time (hrs)	48.2	6.7

4) Metric Definitions

For transparency, we define the principal metrics as computed from CI/CD logs and SARIF artefacts:

- *Critical vulnerabilities pre-prod*: count of unique findings with CVSS $\geq$ 9.0 or vendor severity "Critical" that were detected by any Layer 1–3 scanner before the corresponding build reached the production track.
- *Mean time to deployment (MTTD)*: $\mathrm{MTTD} = \frac{1}{|D|}\sum_{d\in D}(t_{\mathrm{deployed}}(d) - t_{\mathrm{committed}}(d))$, where D is the set of production deployments and timestamps are taken from the CI/CD platform's API.
- *Deployment frequency*: $\mathrm{DF} = \frac{|D|}{w}$, where w is the number of weeks in the period.
- *Failed deployment rate*: fraction of production deployments that triggered automated rollback within 24 hours.
- *Mean detection time*: time from commit to first scanner finding for the vulnerability that the build later triggers a gate failure on (or to first runtime detection if the vulnerability reaches production undetected by static scans).
- *False positive rate*: fraction of findings that were dismissed as "not actionable" during human triage in the PR review process; only the most senior security engineer's adjudication was counted to avoid double-coding bias.

The twelve repositories collectively received 4,312 commits, processed 2,847 pull requests, and triggered 8,924 CI pipeline runs during the 26-week study period.

B. VULNERABILITY DETECTION RESULTS

Table 6 presents the vulnerability detection comparison between baseline and ASCEND pipelines.

ASCEND reduced critical pre-production vulnerabilities by 83.0 percent (from 47 to 8) and high-severity vulnerabilities by 77.8 percent (from 189 to 42). Vulnerabilities reaching production decreased from 23 to 5 (78.3 percent reduction), consistent with the framework's stated objective of blocking vulnerable code at QG1–QG3. Mean detection time decreased from 72.4 hours (manual review cycle) to 18 minutes (0.3 hours), which represents a 99.6 percent improvement attributable to build-gated scanning that executes within minutes of code commit.

The false positive rate decreased from 12.8 percent to 8.2 percent through multi-tool cross-validation: when two

TABLE 7. Deployment efficiency: baseline vs. ASCEND.

Metric	Baseline	ASCEND
MTTD (hours)	18.6	10.5
Deploy freq. (per week)	3.2	8.7
Failed deploys (%)	14.3	4.1
Rollback freq. (%)	8.7	2.3
Security-blocked (%)	6.2	22.8

or more tools independently flag the same code location, confidence is high; when only one tool flags it, the result is queued for human triage rather than blocking the build. Average remediation time decreased from 48.2 hours to 6.7 hours because developers receive vulnerability reports inline in pull requests with suggested fixes rather than through delayed security team reviews.

C. DEPLOYMENT EFFICIENCY RESULTS

Table 7 presents deployment efficiency metrics that compare baseline and ASCEND pipelines.

Mean time to deployment decreased by 43.5 percent, from 18.6 to 10.5 hours. Counterintuitively, deployment frequency *increased* by 171.9 percent (from 3.2 to 8.7 deployments per week) despite stricter quality gates. This improvement is explained by the elimination of manual security review bottlenecks: previously, code awaited asynchronous security review for an average of 11.4 hours before deployment approval, whereas the automated gates in ASCEND provide approval or rejection within 12 minutes. Failed deployments decreased from 14.3 percent to 4.1 percent and rollback frequency decreased from 8.7 percent to 2.3 percent, which reflects improved pre-deployment quality validation.

The security-blocked build rate *increased* from 6.2 percent to 22.8 percent, which represents a positive outcome: builds that previously reached production with undetected vulnerabilities are now caught earlier. This metric reflects the framework's effectiveness at enforcing quality standards rather than its impact on developer productivity.

D. AI CONFLICT RESOLUTION PERFORMANCE

The AI synchronization layer processed 1,847 merge conflicts during the 26-week evaluation period. Before reporting the performance numbers we make the evaluation protocol explicit, since the headline 94.2 percent accuracy figure is sensitive to how "correctly resolved" is defined.

1) Ground-Truth Definition

For each AI-proposed resolution that exceeded the auto-resolve confidence threshold ($\text{conf} \geq 0.85$), the system created a pull request with the proposed code. The original conflict's resolving developer—i.e., the engineer who would have manually resolved the conflict in the absence of ASCEND—reviewed the PR and either merged it as-is, merged it with edits, or rejected it. We define the per-conflict outcome as follows:

- *Correctly resolved*: the developer merged the AI's resolution with no semantic edits. Cosmetic edits (variable renaming, comment rewording, whitespace) were classified as "correct" because they do not change program behavior. The classification rule is encoded in a deterministic AST-equivalence script that runs over the diff between the AI proposal and the merged commit.
- *Incorrect*: the developer made a non-cosmetic change to the AI proposal, or rejected it outright in favor of writing the resolution from scratch.
- *Inconclusive*: the PR remained open at study end (3 cases). These are excluded from the accuracy denominator.

TABLE 8. AI conflict resolution (26-week evaluation).

Metric	Value
Total conflicts	1,847
Auto-resolved (conf. ≥ 0.85)	1,284 (69.5%)
Correctly resolved (auto subset)	1,209 (94.2% of auto)
Inter-rater agreement (κ, $n = 100$)	0.87
Manual intervention (conf. < 0.85)	563 (30.5%)
Avg. AI resolution time	4.2 min
Avg. manual resolution time	47.3 min
Drift incidents	342
Drift auto-remediated	287 (83.9%)

2) Inter-Rater Reliability

To validate the deterministic AST-equivalence rule, two senior engineers independently labelled a stratified random sample of 100 AI-resolved conflicts (25 per conflict type) using the same definition. Cohen's $\kappa = 0.87$ (substantial agreement); the four disagreements were all on the boundary between cosmetic and semantic edits, and were resolved by adjudication with a third reviewer. The agreement statistic supports the use of the automated rule on the full corpus.

3) Treatment of Manual-Intervention Cases

The 30.5 percent of conflicts whose initial AI confidence fell below the 0.85 threshold were escalated to the developer with a notification but *without* an AI proposal. These conflicts followed the participating organization's standard manual merge process. Their resolution time contributes to the "avg. manual time" baseline of 47.3 minutes in Table 8 but does not contribute to the 94.2 percent accuracy figure, which is conditional on auto-resolution. The accuracy of the AI on the manual subset, had it been allowed to propose resolutions there, is therefore not measured by this study; the 94.2 percent figure should be read as "of the conflicts ASCEND chose to auto-resolve, this fraction were accepted with no semantic edits."

The system auto-resolved 69.5 percent of conflicts (those with confidence ≥ 0.85) and achieved 94.2 percent accuracy on this subset. The remaining 30.5 percent required manual intervention, which represents cases where the model's confidence fell below threshold (typically structural conflicts that involve architectural changes). Average AI resolution time of 4.2 minutes compares favorably to 47.3 minutes for manual resolution, an $11.2\times$ speedup. The system correctly identified

TABLE 9. Per-stack vulnerability reduction and MTTD.

Stack	N	Vuln	MTTD	FP
Java Spring	3	81.4%	44.2%	7.8%
Python/Flask	2	79.6%	41.8%	9.1%
Node/React	3	84.3%	45.7%	8.4%
Go	1	77.2%	38.6%	6.2%
Terraform/K8s*	3	88.9%	42.1%	5.7%
All	12	82.3%	43.5%	7.9%

* IaC repos have higher baseline misconfig. rates.

and auto-remediated 83.9 percent of drift incidents.

Accuracy varied by conflict type: syntactic conflicts at 97.1 percent, configuration conflicts at 95.6 percent, semantic conflicts at 91.8 percent, and structural conflicts at 87.3 percent. The lower accuracy on structural conflicts is expected, because they involve reasoning about architectural intent that is difficult to infer from code context alone.

E. LONGITUDINAL TRENDS

Tracking the weekly trend in security-blocked builds and deployment frequency over the 26-week study period reveals a transition phase (weeks 14–20) with rapid improvement in both metrics as teams calibrate scanning configurations and quality gate thresholds. Both metrics stabilize by week 20, which suggests that a 6 to 7 week adoption ramp-up period is typical for organizations of this scale.

F. PER-REPOSITORY ANALYSIS

Table 9 presents vulnerability reduction and mean time to deployment improvement broken down by repository technology stack, which provides actionable guidance for organizations deploying ASCEND to specific technology ecosystems.

Node.js and React repositories showed the highest vulnerability reduction (84.3 percent), attributable to the prevalence of npm dependency vulnerabilities detected by SCA scanning. Terraform and Kubernetes repositories showed the highest reduction (88.9 percent) when considering IaC misconfigurations, which reflects the large proportion of Center for Internet Security benchmark violations detectable by Checkov. Go exhibited the lowest false positive rate (6.2 percent) due to Go's strong typing and limited ambiguity in static analysis.

G. STATISTICAL ANALYSIS

Because the same twelve repositories are observed in both the baseline and treatment periods, the design is within-subjects, and we apply a paired t-test on per-repository pre/post means. We additionally report the non-parametric Wilcoxon signed-rank statistic as a robustness check; all five primary tests reject the null hypothesis under both procedures. Bonferroni correction is applied for multiple comparisons ($\alpha_{\text{corrected}} = 0.05/5 = 0.01$). Metrics are reported as means over the 26-week evaluation period, with standard deviation computed across the twelve repositories. Table 10 presents the complete statistical analysis.

TABLE 10. Statistical analysis ($n = 12$ repositories, paired t-test, $\alpha_{\text{corrected}} = 0.01$).

Metric	t	p	d	95% CI
Critical vuln. red.	9.4	< 0.001	2.72	[2.13, 3.31]
MTTD reduction	7.8	< 0.001	2.25	[1.71, 2.79]
Deploy freq. incr.	11.2	< 0.001	3.24	[2.58, 3.90]
Failed deploy red.	6.9	< 0.001	1.99	[1.49, 2.49]
AI resol. accuracy	8.3	< 0.001	2.40	[1.84, 2.96]

TABLE 11. Cohen's d interpretation bands [29] and ASCEND metric placement.

Range	Interpretation	ASCEND metrics
$0.2 \leq d < 0.5$	small	—
$0.5 \leq d < 0.8$	medium	—
$0.8 \leq d < 1.2$	large	—
$1.2 \leq d < 2.0$	very large	—
$d \geq 2.0$	huge / categorical	all five primary metrics

Confidence intervals on Cohen's d are reported using the bootstrap procedure of Hedges and Olkin with 10,000 resamples; all intervals exclude both zero and the conventional "large effect" threshold of 0.8 [29]. To place the magnitudes in interpretive context, Table 11 reproduces the standard Cohen bands and locates each ASCEND metric.

All comparisons are significant after Bonferroni correction, with very large effect sizes (Cohen's $d > 2.0$ [29]) across all metrics. Effect sizes of this magnitude are unusual in observational software engineering studies; they reflect the categorical difference between manual asynchronous security review and automated build-gated review, which differ in kind rather than in degree. Smaller effects would be expected when comparing two automated frameworks against each other rather than against a manual baseline. We therefore report effect sizes alongside p-values to support transparent interpretation rather than to claim that ASCEND's intervention produces $> 2\sigma$ shifts in every operating context.

IX. THREATS TO VALIDITY

Internal Validity

The pre/post design introduces the threat that improvements are attributable to factors other than ASCEND, such as team maturity or increased security awareness due to study participation (the Hawthorne effect [30]). We mitigated this threat by maintaining identical team composition, sprint processes, and business requirements across both periods. However, we cannot fully exclude the possibility that teams altered behavior due to awareness of the evaluation.

External Validity

The study encompassed twelve repositories across three organizations, which may not represent the full diversity of enterprise software ecosystems. Organizations with highly regulated codebases (healthcare, defense), legacy mainframe systems, or exclusively proprietary tooling stacks may experience different results. The six-month study period may not capture seasonal deployment patterns (such as holiday freezes) or technology transitions.

Construct Validity

The primary security metric (vulnerability count) depends on the detection coverage of the integrated scanning tools. Tool updates between the baseline and treatment periods could introduce differences attributable to tool improvement rather than to the ASCEND framework. We mitigated this risk by pinning all tool versions throughout the study. The AI conflict resolution accuracy metric is measured on auto-resolved conflicts only; accuracy on the 30.5 percent of conflicts escalated to manual resolution is not captured.

Statistical Conclusion Validity

The study uses a within-subjects design with $n = 12$ repositories, which limits statistical power for detecting small effects. All reported metrics include Cohen's d to characterize practical significance beyond p-values. The Bonferroni correction is conservative; less stringent corrections (such as the Benjamini-Hochberg procedure) would yield the same conclusions.

X. DISCUSSION

A. KEY FINDINGS

ASCEND demonstrates that integrating security scanning as an active build gate—rather than a reporting layer—produces measurably superior security outcomes. The 99.6 percent reduction in mean detection time from 72 hours to 18 minutes has significant implications for the cost of remediation: NIST guidance and prior software-economics analyses consistently show that vulnerability remediation costs increase by a factor of 10 to 100 times at each development lifecycle stage [31]. By catching vulnerabilities at commit time rather than in production, ASCEND reduces remediation cost by an estimated 6.7× based on the observed remediation time ratio (48.2 versus 6.7 hours).

The counterintuitive increase in deployment frequency (171.9 percent) despite stricter quality gates aligns with the findings of Forsgren *et al.* [32], who identified that high-performing engineering teams simultaneously achieve faster deployments and fewer failures, which contradicts the traditional speed-quality trade-off assumption. ASCEND automates security review, which replaces a serializing bottleneck with a parallel automated process and decouples deployment throughput from security review capacity.

B. PLATFORM SELECTION GUIDANCE

Based on the platform comparative analysis (Tables 2 and 4), we summarize platform fit against three criteria: integration cost, customization headroom, and governance footprint. *GitHub Actions* is appropriate when the source repository is already on GitHub and CodeQL integration is required at zero marginal cost. *GitLab CI/CD* requires the smallest bespoke configuration owing to native security templates and is appropriate when rapid initial adoption is the dominant constraint. *Jenkins* is appropriate when an existing Jenkins controller must be retained and custom integrations dominate over baseline capability coverage. *Azure DevOps* is appropriate when role-based access control, native Microsoft directory integration, and centrally administered audit trails are governance requirements rather than nice-to-haves.

TABLE 12. ASCEND compliance alignment.

Framework	Requirement	ASCEND
PCI DSS 6.3.2	Code inventory	SCA (L1)
PCI DSS 6.4.1	Web app prot.	DAST (L3)
SOC 2 II CC8.1	Change mgmt	Quality gates
SOC 2 II CC7.1	Threat detect.	SAST (L1)
NIST CSF ID.RA-1	Asset vulns	SCA, IaC
NIST CSF PR.DS-6	Integrity	Secrets
ISO 27001 A.14.2.2	Sec. dev	All layers
HIPAA Rule 164.312	Audit	Pipeline logs

C. REGULATORY COMPLIANCE ALIGNMENT

ASCEND's quality gate framework directly supports compliance with major information security regulations and frameworks. Table 12 maps ASCEND components to compliance requirements across four major frameworks.

The automated audit trail generated by ASCEND—scan results, quality gate outcomes, approval records, and deployment timestamps—provides the evidence artifacts required for SOC 2 Type II audits without additional compliance tooling. Organizations subject to the Payment Card Industry Data Security Standard 4.0 can use ASCEND DAST scanning (Requirement 6.4.1) and code inventory (Requirement 6.3.2) to satisfy web application protection requirements that previously required manual penetration testing.

D. COST-BENEFIT ANALYSIS

The total implementation cost of ASCEND includes three components: (1) tooling costs (scanning tool licenses where applicable, such as SonarQube Enterprise and Snyk Team); (2) CI infrastructure overhead (additional compute for parallel scanning); and (3) engineering time for initial setup and calibration.

Based on the observed remediation time reduction from 48.2 to 6.7 hours per vulnerability and industry data on developer hourly costs (USD 150 to USD 200 per hour for senior engineers), the cost savings per vulnerability remediated is approximated by Eq. (7):

$$\Delta \mathrm{Cost}_{\mathrm{vuln}} = (48.2 - 6.7) \times 175 = \$7{,}262. \tag{7}$$

Across the twelve repositories in our evaluation, ASCEND prevented 283 vulnerabilities from requiring late-stage remediation (baseline: 236 total pre-production vulnerabilities; ASCEND: 50 total), which yields estimated savings of approximately USD 2.05 million over the 26-week study period. The annual break-even for ASCEND tooling costs (approximately USD 80 000 for enterprise tooling licenses) is reached with as few as 11 prevented late-stage vulnerabilities per year, well below the observed prevention rate.

E. ADOPTION ROADMAP

ASCEND's layered architecture enables incremental adoption. Organizations can implement Layer 1 (source scan-

ning) in 1 to 2 weeks, achieving the majority of security improvement (83 percent of vulnerability reduction) before investing in the more complex deployment orchestration and AI synchronization layers. Layers 2 and 3 typically require 4 to 6 additional weeks. Layer 4 (AI synchronization) requires the longest investment—8 to 12 weeks for model training and calibration—but yields the largest absolute reduction in manual conflict-resolution time for organizations with frequent merge conflicts and multi-track deployments (4.2 versus 47.3 minutes per conflict in this study).

F. LIMITATIONS

The AI synchronization layer requires a historical corpus of resolved merge conflicts for model training; organizations with fewer than 500 historical conflicts may find the model insufficiently specialized. The framework's platform configurations have been validated on the four major platforms evaluated; other CI/CD systems (CircleCI, TeamCity, Bamboo) would require analogous but platform-specific configuration adaptations. Container scanning is limited to Open Container Initiative compatible registries and does not cover virtual-machine-based deployments.

G. FUTURE WORK

Future directions include (1) runtime application self-protection feedback integration, which closes the loop between production runtime anomalies and CI/CD pipeline scanning policies; (2) polyglot AI conflict resolution that supports cross-language dependency analysis; (3) federated learning to allow organizations to improve the shared conflict resolution model without exposing proprietary code; (4) integration with software bill of materials standards (CycloneDX, SPDX) for supply chain security visibility; and (5) reinforcement learning for dynamic quality gate threshold adaptation based on deployment risk signals.

XI. CONCLUSION

We have presented ASCEND, a four-layer DevSecOps framework that treats security scanning as an active build gate rather than a post-build report. The framework covers SAST, DAST, SCA, and IaC scanning across GitHub Actions, GitLab CI/CD, Jenkins, and Azure DevOps, with configurable quality gates between layers. Experimental evaluation across twelve enterprise repositories over six months demonstrates statistically significant improvements across all primary metrics: an 83.0 percent reduction in critical pre-production vulnerabilities ($p < 0.001$, Cohen's $d = 2.72$), a 43.5 percent decrease in mean time to deployment ($d = 2.25$), a 171.9 percent increase in deployment frequency ($d = 3.24$), and 94.2 percent accuracy on the auto-resolved subset of conflicts ($d = 2.40$). The AI synchronization layer auto-remediated 83.9 percent of detected drift incidents, which significantly reduces the manual effort required to maintain code consistency across multi-track deployment environments.

The multi-tool scanning design achieves 96 percent combined vulnerability detection compared with 67 to 71 percent for any individual tool, and the AI synchronization layer delivers an 11.2× speedup in conflict resolution (4.2 versus 47.3 minutes). The platform-agnostic architecture spans GitHub Actions, GitLab CI/CD, Jenkins, and Azure DevOps, and the layered design supports incremental adoption with meaningful security gains achievable through Layer 1 alone.

ETHICS STATEMENT

The 26-week study was conducted as observational research over engineering metrics already collected by each participating organization's CI/CD platform. No personally identifiable information about contributors was collected, and the data-handling pipeline (Section VIII) suppresses any aggregate cell with $k < 5$ underlying records. Each participating organization's data-protection office reviewed and approved the protocol. Because the unit of analysis is the repository rather than any individual person, the study was determined to be exempt from human-subjects research review by the participating organizations' internal review processes.

DATA AVAILABILITY STATEMENT

The ASCEND framework—platform configurations for all four CI/CD systems, the AI synchronization module source, the conflict-fixtures benchmark suite, and the experimental analysis pipeline—is openly available at https://github.com/venkatapgummadi/ascend under the MIT license. The companion analysis script `evaluation/statistics.py` reproduces Table 10 byte-for-byte from a per-repository, per-week, k-anonymized CSV in the documented schema. Per-repository telemetry from the 26-week study cannot be released due to organizational data-handling agreements; restricted-access reproductions are available on request to the corresponding author, subject to the participating organizations' data-sharing terms.

REPRODUCIBILITY NOTE

A reviewer-facing checklist that maps each citation key to its archival source, lists code-vs-paper alignment notes, and provides pre-resubmission verification commands is included in the open-source repository at `docs/paper/REVIEWER_CHECKLIST.md`. The extended methodology document at `docs/paper/EVALUATION.md` provides the variable definitions, anonymization pipeline, and threats-to-validity expansion that did not fit within the page budget of this article.

REFERENCES

[1] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," *Information and Software Technology*, vol. 141, p. 106700, 2022.

[2] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*. ACM, 2016, pp. 426–437.

[3] G. Ghiotto, L. Murta, M. Barros, and A. van der Hoek, "On the nature of merge conflicts: A study of 2,731 open source Java projects hosted by GitHub," *IEEE Transactions on Software Engineering*, vol. 46, no. 8, pp. 892–915, 2020.

[4] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

[5] L. Chen, "Continuous delivery: Huge benefits, but challenges too," *IEEE Software*, vol. 32, no. 2, pp. 50–54, 2015.

[6] B. Shen, W. Zhang, H. Zhao, G. Liang, Z. Jin, and Q. Wang, "IntelliMerge: A refactoring-aware software merging technique," *Proc. ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 170:1–170:28, 2019.

[7] A. Svyatkovskiy, S. Fakhoury, N. Ghorbani, T. Mytkowicz, E. Dinella, C. Bird, J. Jang, N. Sundaresan, and S. K. Lahiri, "Program merge conflict resolution via neural transformers," in *Proc. 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2022, pp. 822–833.

[8] E. Dinella, T. Mytkowicz, A. Svyatkovskiy, C. Bird, M. Naik, and S. K. Lahiri, "DeepMerge: Learning to merge programs," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1599–1614, 2023.

[9] H. Myrbakken and R. Colomo-Palacios, "DevSecOps: A multivocal literature review," in *Software Process Improvement and Capability Determination (PROFES)*. Springer, 2017, pp. 17–29.

[10] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015.

[11] T. Rangnau, R. v. Buijtenen, F. Fransen, and F. Turkmen, "Continuous security testing: A case study on integrating dynamic security testing tools in CI/CD pipelines," in *Proc. 24th Int. Conf. Engineering of Complex Computer Systems (ICECCS)*. IEEE, 2020, pp. 145–154.

[12] A. Gosain and G. Sharma, "A survey of dynamic program analysis techniques and tools," in *Proc. 3rd Int. Conf. Frontiers of Intelligent Computing: Theory and Applications (FICTA)*. Springer, 2015, pp. 113–122.

[13] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, and M. Di Penta, "How open source projects use static code analysis tools in continuous integration pipelines," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*. IEEE, 2017, pp. 334–344.

[14] A. Habib and M. Pradel, "How many of all bugs do we find? a study of static bug detectors," in *Proc. 33rd ACM/IEEE Int. Conf. Automated Software Engineering (ASE)*. ACM, 2018, pp. 317–328.

[15] N. Antunes and M. Vieira, "Comparing the effectiveness of penetration testing and static code analysis on the detection of SQL injection vulnerabilities in web services," in *Proc. 16th IEEE Pacific Rim Int. Symp. Dependable Computing (PRDC)*. IEEE, 2010, pp. 157–166.

[16] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, "Do developers update their library dependencies? an empirical study on the impact of security advisories on library migration," *Empirical Software Engineering*, vol. 23, no. 1, pp. 384–417, 2018.

[17] A. Decan, T. Mens, and E. Constantinou, "On the evolution of technical lag in the npm package dependency network," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*. IEEE, 2018, pp. 404–414.

[18] D. Everson, L. Cheng, and Z. Zhang, "Log4Shell: Redefining the web attack surface," in *Proc. Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*. Internet Society, 2022.

[19] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. Security and Privacy (S&P)*. IEEE, 2022, pp. 754–768.

[20] R. Tufano, S. Masiero, A. Mastropaolo, S. Scalabrino, R. Oliveto, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proc. 44th Int. Conf. Software Engineering (ICSE)*. ACM, 2022, pp. 2291–2302.

[21] L. Leite, C. Rocha, F. Kon, D. Milojicic, and P. Meirelles, "A survey of DevOps concepts and challenges," *ACM Computing Surveys*, vol. 52, no. 6, pp. 1–35, 2020.

[22] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-world challenges and research innovations," in *Proc. 41st Int. Conf. Software Engineering: Companion (ICSE-Companion)*. IEEE, 2019, pp. 4–5.

[23] T. Sandall, "Open Policy Agent: A policy-based control-plane for cloud-native environments," in *KubeCon + CloudNativeCon Industry Track*, 2018, cloud Native Computing Foundation graduated project; specification at https://www.openpolicyagent.org/.

[24] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "Source code properties of defective infrastructure as code scripts," *Information and Software Technology*, vol. 112, pp. 148–163, 2019.

[25] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proc. 41st Int. Conf. Software Engineering (ICSE)*. IEEE, 2019, pp. 164–175.

[26] M. Souppaya, K. Scarfone, and D. Dodson, "Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-218, 2022.

[27] G. Schermann, J. Cito, P. Leitner, U. Zdun, and H. C. Gall, "We're doing it live: A multi-method empirical study on continuous experimentation," *Information and Software Technology*, vol. 99, pp. 41–57, 2018.

[28] T. Savor, M. Douglas, M. Gentili, L. Williams, K. Beck, and M. Stumm, "Continuous deployment at Facebook and OANDA," in *Proc. 38th Int. Conf. Software Engineering Companion (ICSE-C)*. ACM, 2016, pp. 21–30.

[29] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.

[30] H. A. Landsberger, *Hawthorne Revisited: Management and the Worker, Its Critics, and Developments in Human Relations in Industry*. Cornell University, 1958.

[31] B. W. Boehm, *Software Engineering Economics*. Prentice Hall, 1981.

[32] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, 2018.

VENKATA PAVAN KUMAR GUMMADI (Senior Member, IEEE) is a Professional Software Engineer with more than 18 years of experience in DevSecOps, enterprise integration, and AI-driven automation. His research interests include continuous-integration security architecture, multi-track deployment orchestration, and the integration of large-language-model-based reasoning with traditional software-engineering pipelines. He is the author of the AgentFlow and ASCEND frameworks. He holds an ORCID iD of 0009-0005-4435-0197 and can be reached at `venkata.p.gummadi@ieee.org`.